IMU Based RIR Detection

Josh, Ryan, Evan, Daniel, Colin, Akshay

Video

Project Inspiration

Velocity is a prime indicator of RIR and failure proximity

ORIGINAL RESEARCH

Movement Velocity as a Measure of Level of Effort During Resistance Exercise

Morán-Navarro, Ricardo[1,2]; Martínez-Cava, Alejandro[1]; Sánchez-Medina, Luis[3]; Mora-Rodríguez, Ricardo[2]; González-Badillo, Juan José[4]; Pallarés, Jesús G.[1,2]

Author Information

Journal of Strength and Conditioning Research 33(6):p 1496-1504, June 2019. | ***DOI:*** 10.1519/JSC.0000000000002017

FREE

BMC Sports Sci Med Rehabil. 2025 Mar 25;17:60. doi: 10.1186/s13102-025-01098-2

Gauging proximity to failure in the bench press: generalized velocity-based vs. %1RM-repetitions-to-failure approaches

Xuelin Qin [1], Beibei Liu [2], Amador García-Ramos [3,4]

Author information · Article notes · Copyright and License information

PMCID: PMC11934800 PMID: 40133968

Lifting velocity predicts the maximum number of repetitions to failure with comparable accuracy during the Smith machine and free-weight prone bench pull exercises

Sergio Miras-Moreno [a], Alejandro Pérez-Castilla [b c], Francisco J. Rojas-Ruiz [a], Amador García-Ramos [a d]

Show more

Add to Mendeley · Share · Cite

https://doi.org/10.1016/j.heliyon.2023.e19628

Get rights and content

Under a Creative Commons license

Open access

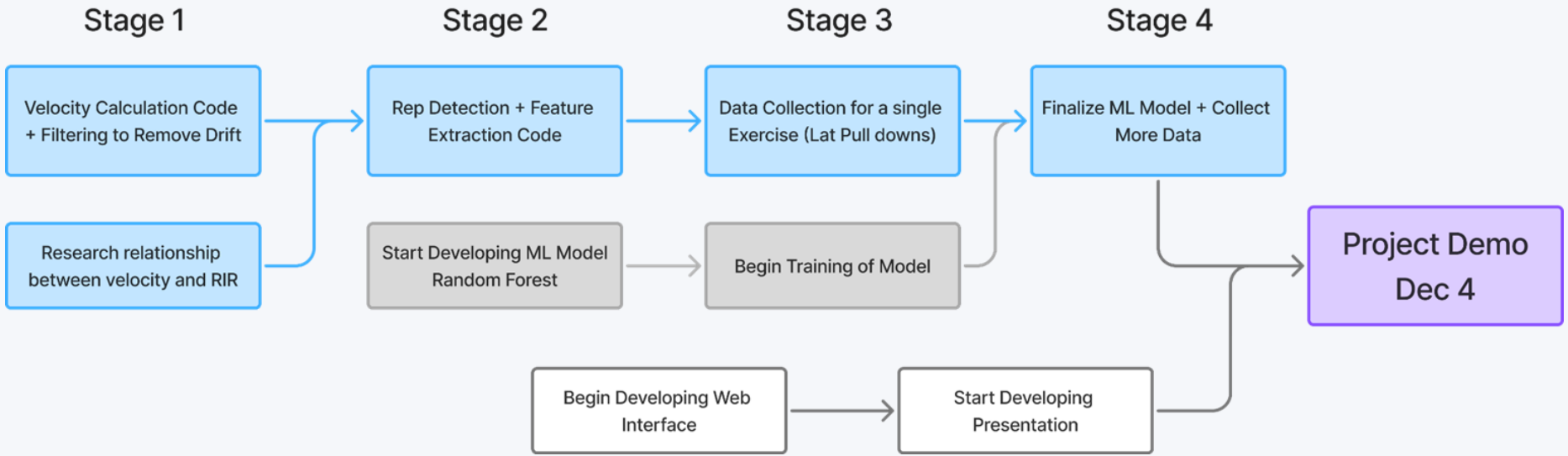
Project Overview

Arduino Nano BLE Sense 33 - Inertial Measurement Unit (IMU)

Arduino Code + ~~MATLAB Code~~ (Pythonified) to calculate velocity and observe a set

Random Forest Regression ML Model in Python

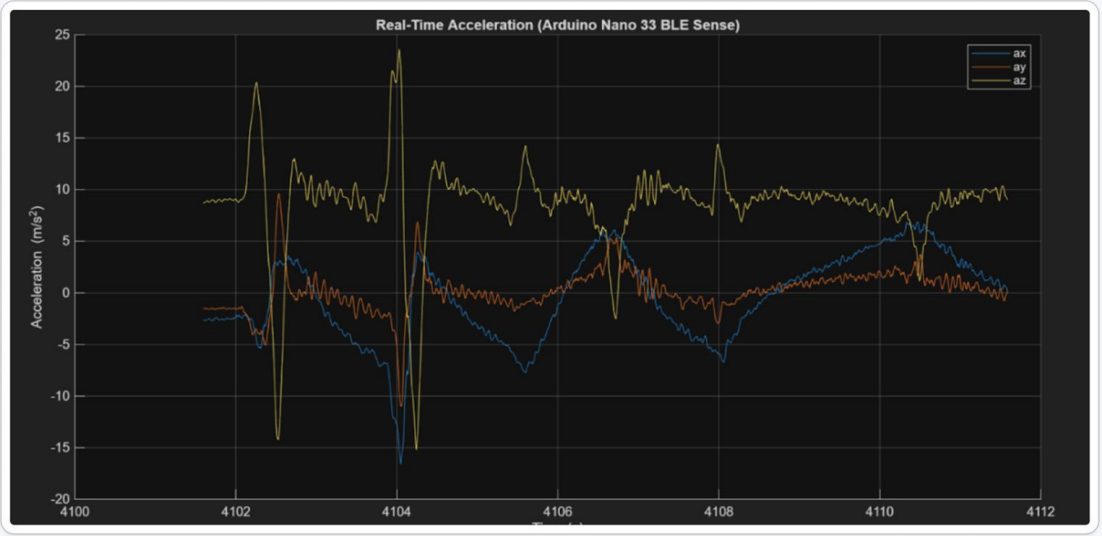
Roadmap and Goals



VELOCITY CALCULATION STAGE 1 + 2

Stage 1 + 2

- IMU always has gravity on Z
- FFT analysis of acceleration data
- Gravity very low frequency noise
- Decided to use a HPF to remove

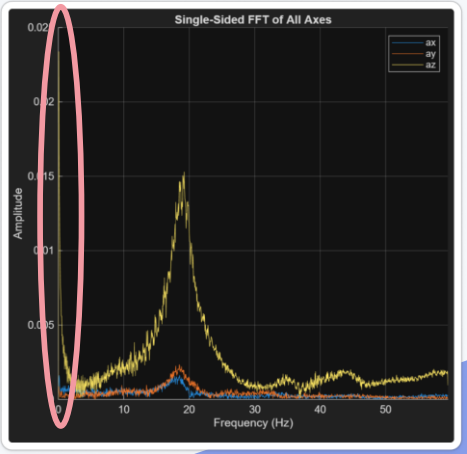



```
if (IMU.accelerationAvailable()) {
  IMU.readAcceleration(ax, ay, az);

  unsigned long t = micros();  // timestamp in microseconds

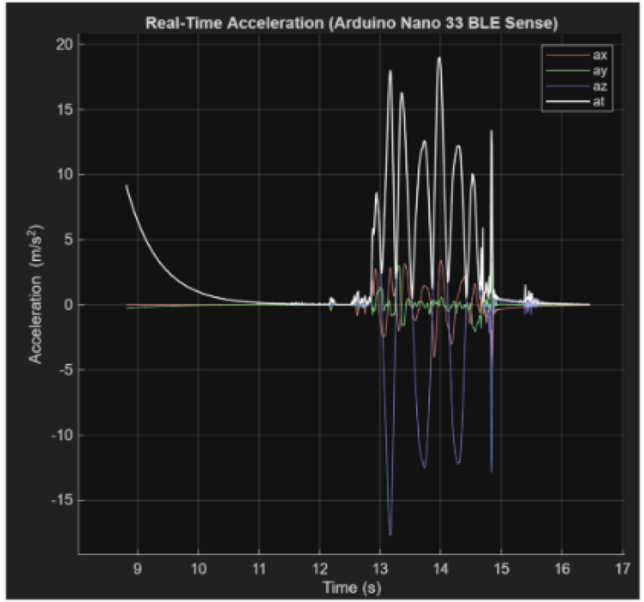
  // ax, ay, az in g's. Convert to m/s^2:
  ax *= 9.80665;
  ay *= 9.80665;
  az *= 9.80665;

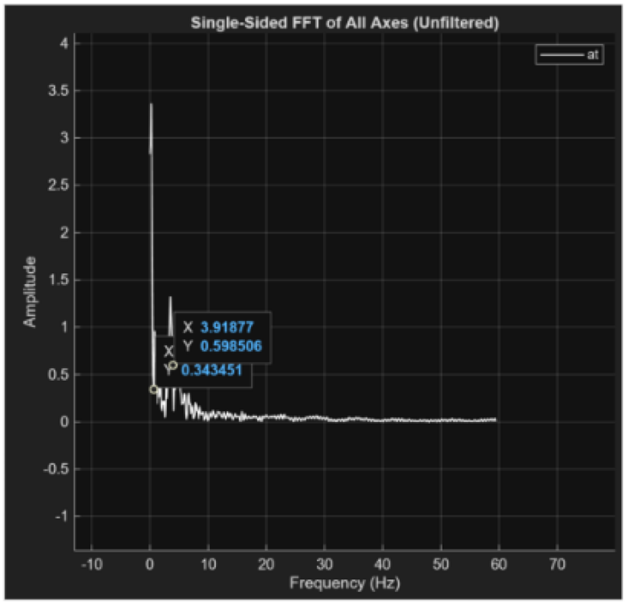

  // CSV output
  Serial.print(t); Serial.print(',');
  Serial.print(ax, 6); Serial.print(',');
  Serial.print(ay, 6); Serial.print(',');
  Serial.println(az, 6);
}
```



Stage 1 + 2

- HPF implemented in the code
- Accelerations converge to 0 when stationary
- Gravity removed / mitigated when stationary






```
float HPF_filter(float x, float &x_prev, float &y_prev) {
  float y = hpf_alpha * (y_prev + x - x_prev);
  x_prev = x;
  y_prev = y;
  return y;
}
```

```
// ------------ HPF VARS ------------
float fs = 119.0; // Sampling Rate
float fc = 0.3; // HPF Cutoff

float Ts = 1.0 / fs; // sampling period
float tau = 1.0 / (2.0 * 3.14159 * fc);
float hpf_alpha = tau / (tau + Ts);
```

Stage 1 + 2

```
// ACCELERATION FILTERING | HPF to remove drift
ax_filtered = HPF_filter(ax, ax_in_prev, ax_out_prev);
ay_filtered = HPF_filter(ay, ay_in_prev, ay_out_prev);
az_filtered = HPF_filter(az, az_in_prev, az_out_prev);

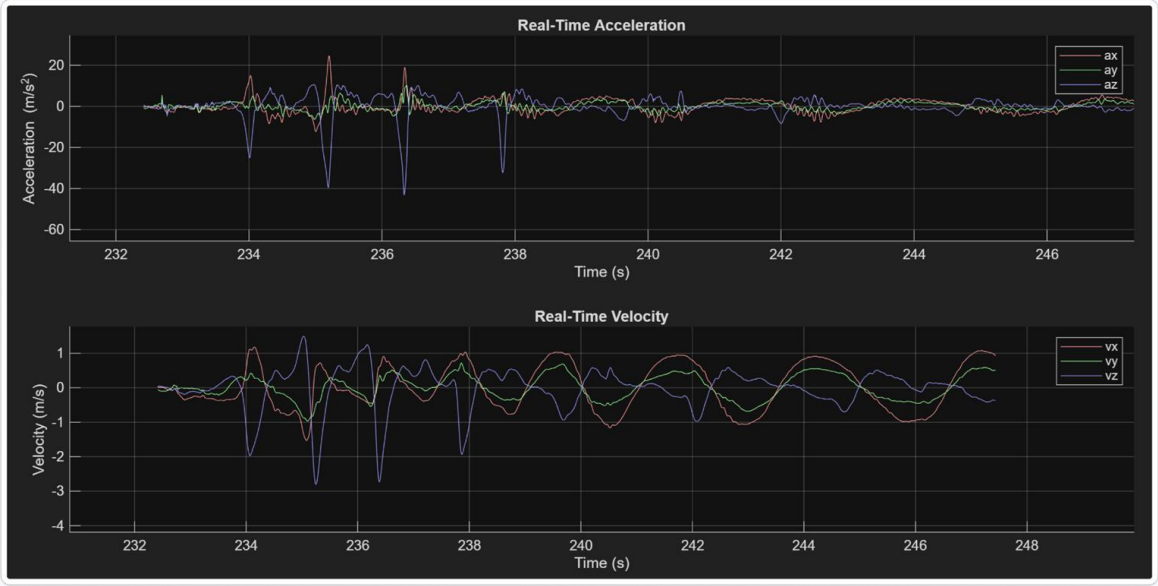
// VELOCITY CALCS
vx += ax_filtered * ((t - lastMicros) * 1e-6);
vy += ay_filtered * ((t - lastMicros) * 1e-6);
vz += az_filtered * ((t - lastMicros) * 1e-6);
lastMicros = t;

// VELOCITY FILTERING
vx_filtered = HPF_filter(vx, vx_in_prev, vx_out_prev);
vy_filtered = HPF_filter(vy, vy_in_prev, vy_out_prev);
vz_filtered = HPF_filter(vz, vz_in_prev, vz_out_prev);

vel_reset(vx); vel_reset(vy); vel_reset(vz);

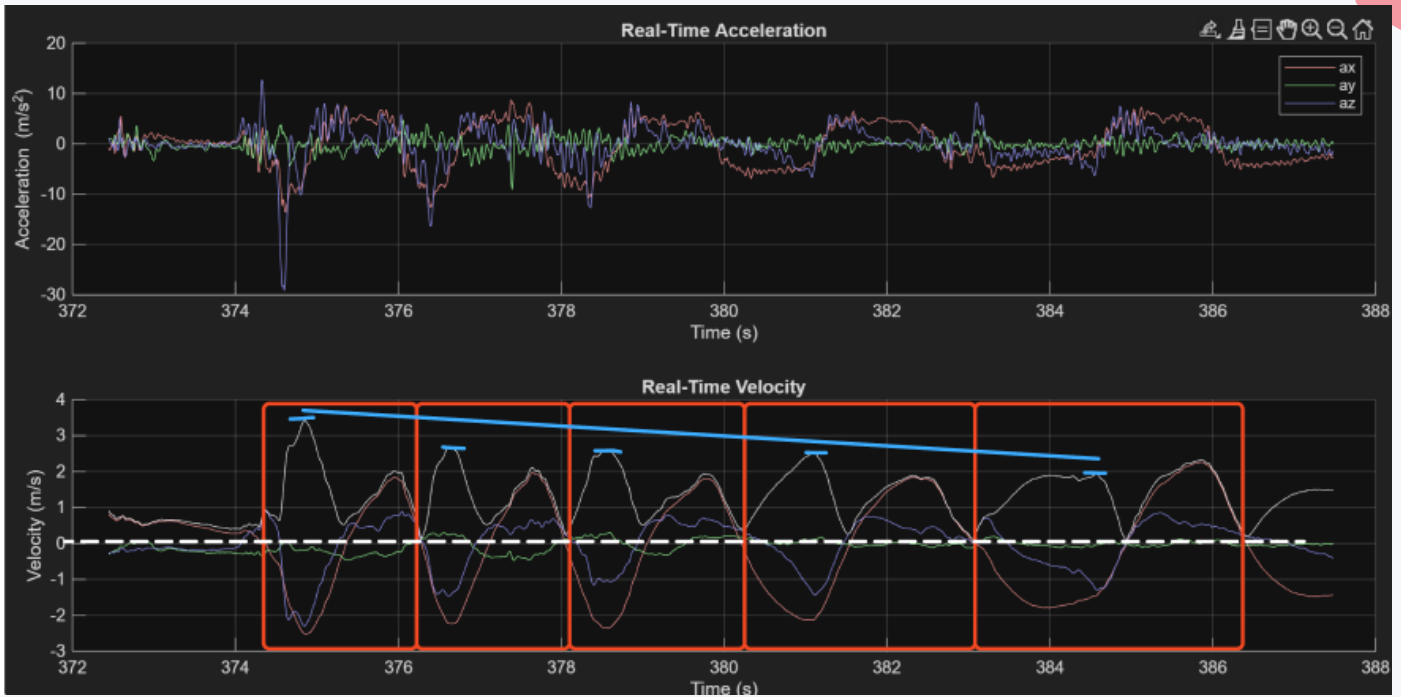

// CSV OUTPUT --- PRINTING ---
Serial.print(t); Serial.print(',');

Serial.print(vx_filtered); Serial.print(',');
Serial.print(vy_filtered); Serial.print(',');
Serial.println(vz_filtered);
```



- Calculated Velocity by integrating over time
- Velocity no longer drifting due to gravity
- More accurate data to work with

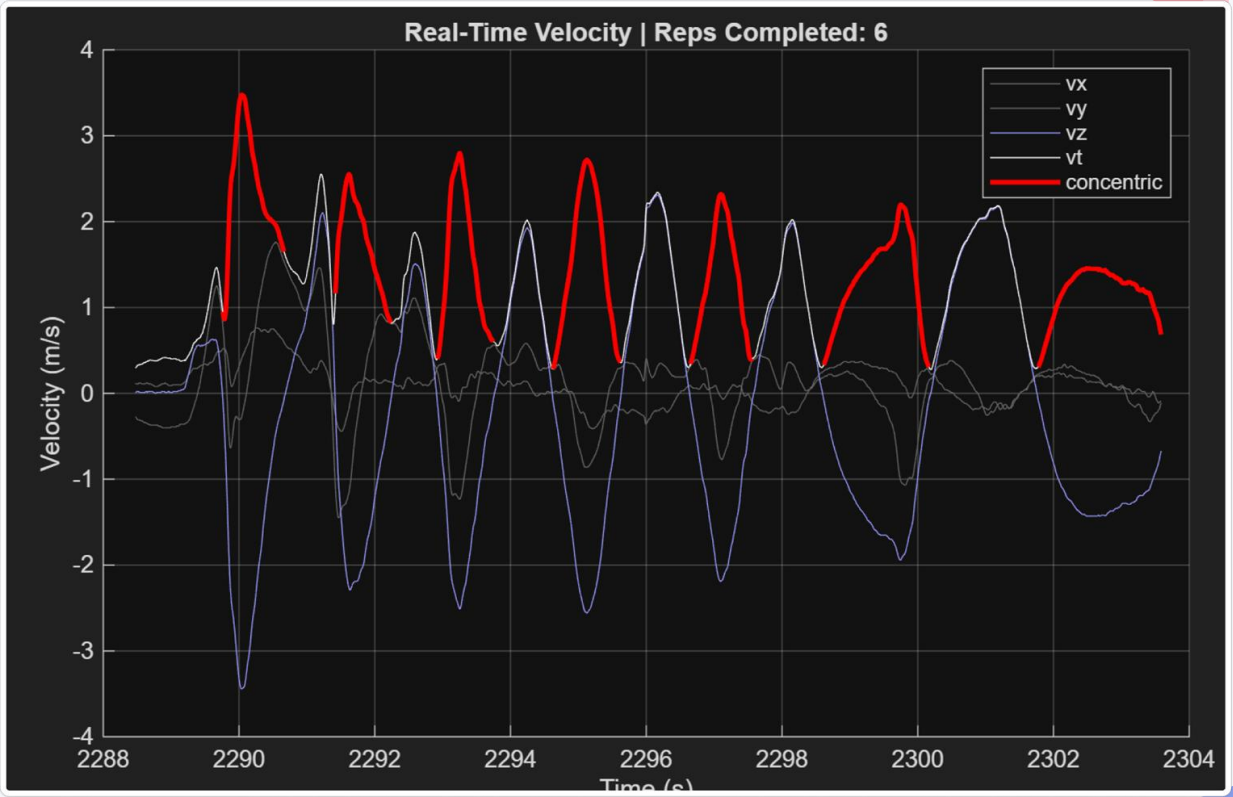
Stage 1 + 2



- Simulated a bicep curl with increased fatigue (slowing)
- Noticed each rep had two peaks (Concentric + Eccentric)
- Decreasing peaks

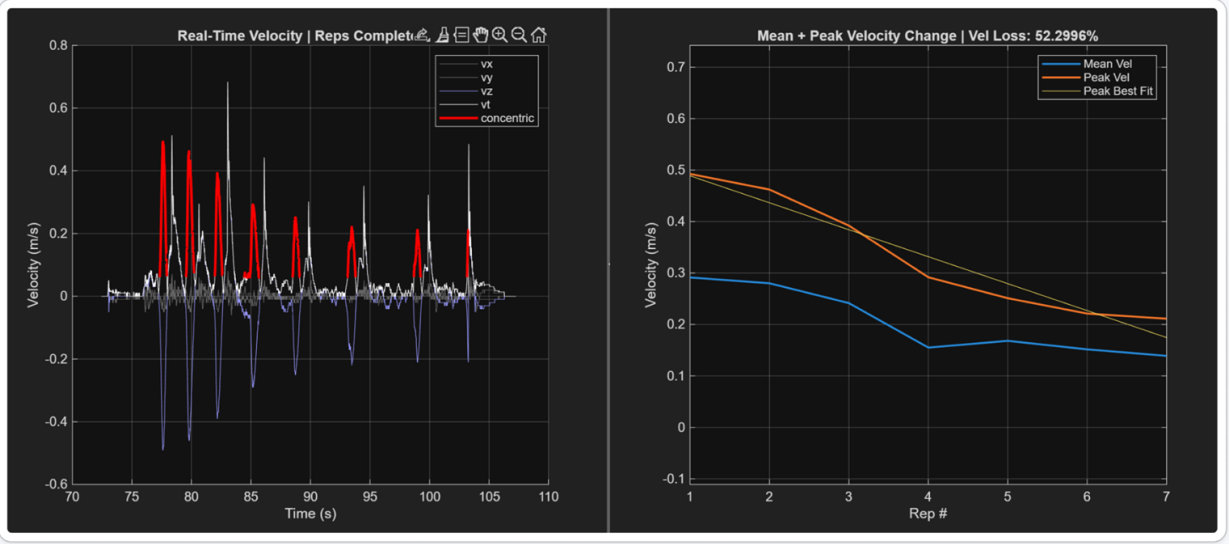
Stage 1 + 2

- When vz < 0, the rep is in a concentric
- Wrote rep detection code for when
  - Vz < -0.05
  - Rep lasts longer than 0.5s
- Stores indices in data for entire window



Stage 1 + 2

- For each rep we calculate:
  - Mean velocity
  - Peak velocity
  - Duration
- Then set level features
  - Best fit line calculated using Least Squares



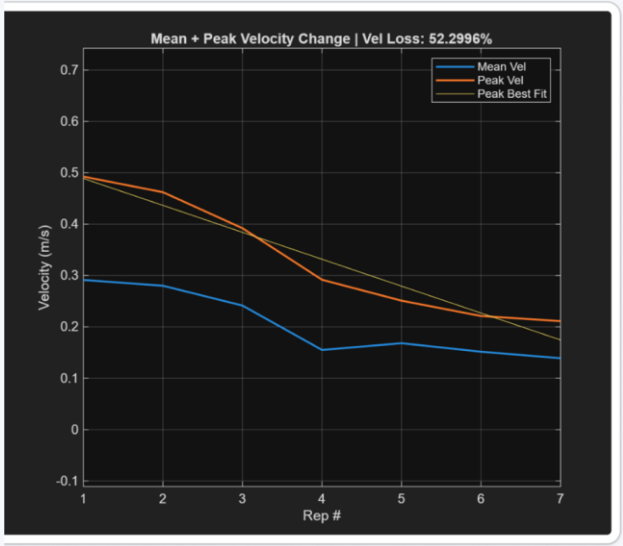
Rep#	Duration (s)	StartIdx	EndIdx	Avg Vel
1	0.55587	100	166	1.5439
2	0.59011	281	351	0.60131
3	0.69924	467	550	0.66034
4	0.84247	656	756	0.5685
5	0.97679	867	983	0.59539
6	0.99398	1102	1220	0.46095
7	1.2127	1342	1486	0.52015
8	0.64838	1608	1685	0.27022

DATA COLLECTION STAGE 3

Stage 3

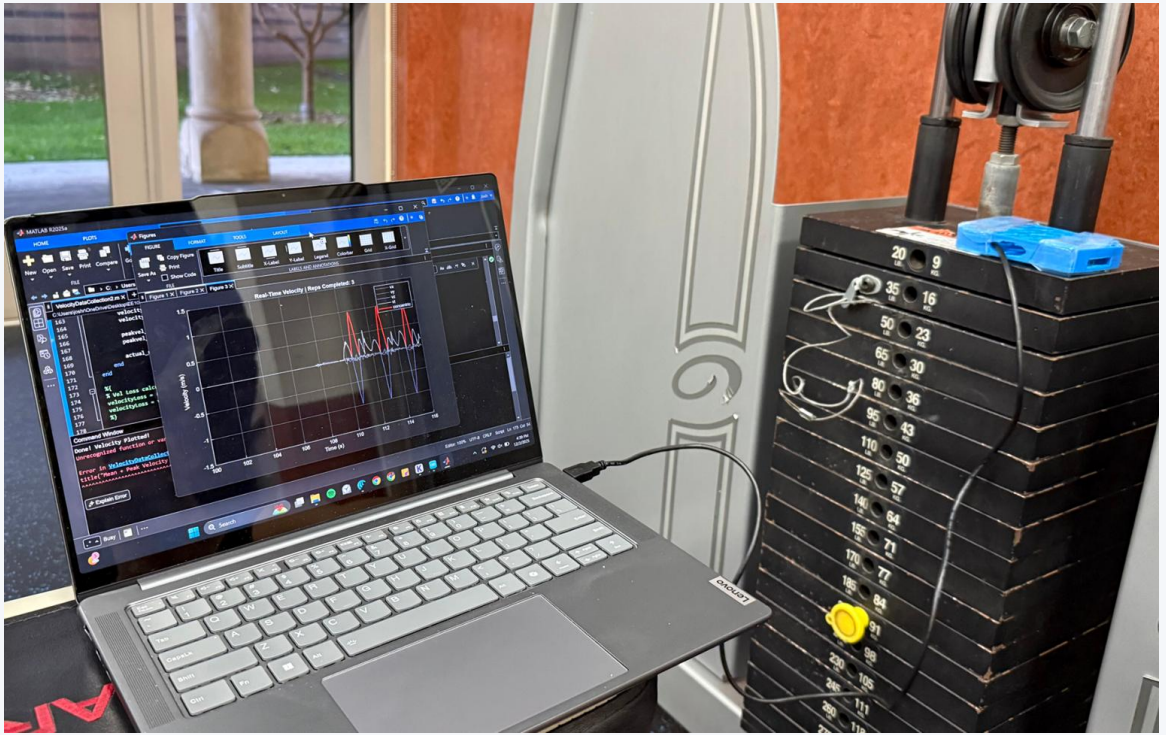
Data Collection Methodology

- Ensuring that only eccentric velocity is accounted for
- Concentric velocity is constant to nullify its influence
- Velocity with a decreasing slope is favorable for model



Stage 3






NAUTILUS


NAUTILUS





Thank you to the following people:

Annika Kumar, Aiden Fox, Natty Yolsirivat, Akshit Gupta, Micah Kim

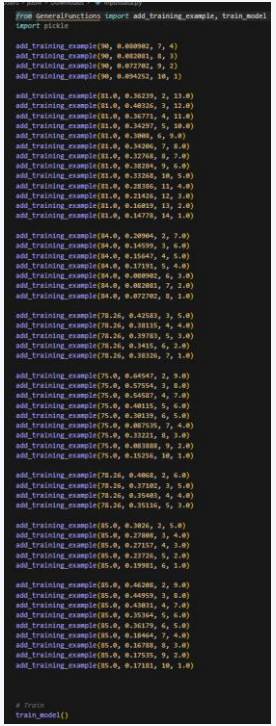
MODEL TRAINING STAGE 4

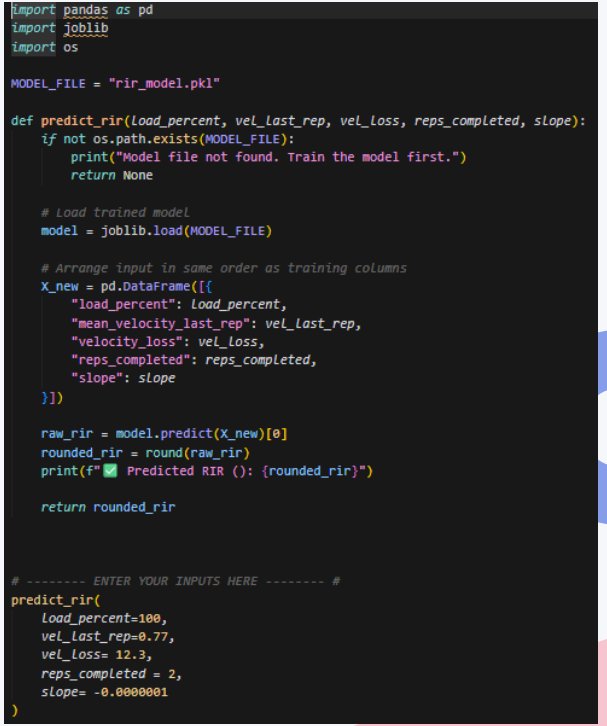
Stage 4

Random Forest Regression Model

- Simpler model for smaller data sets
- Easier training
- No linear regression
- No neural network(likely would have overfitting due to time limit and unable to spread)

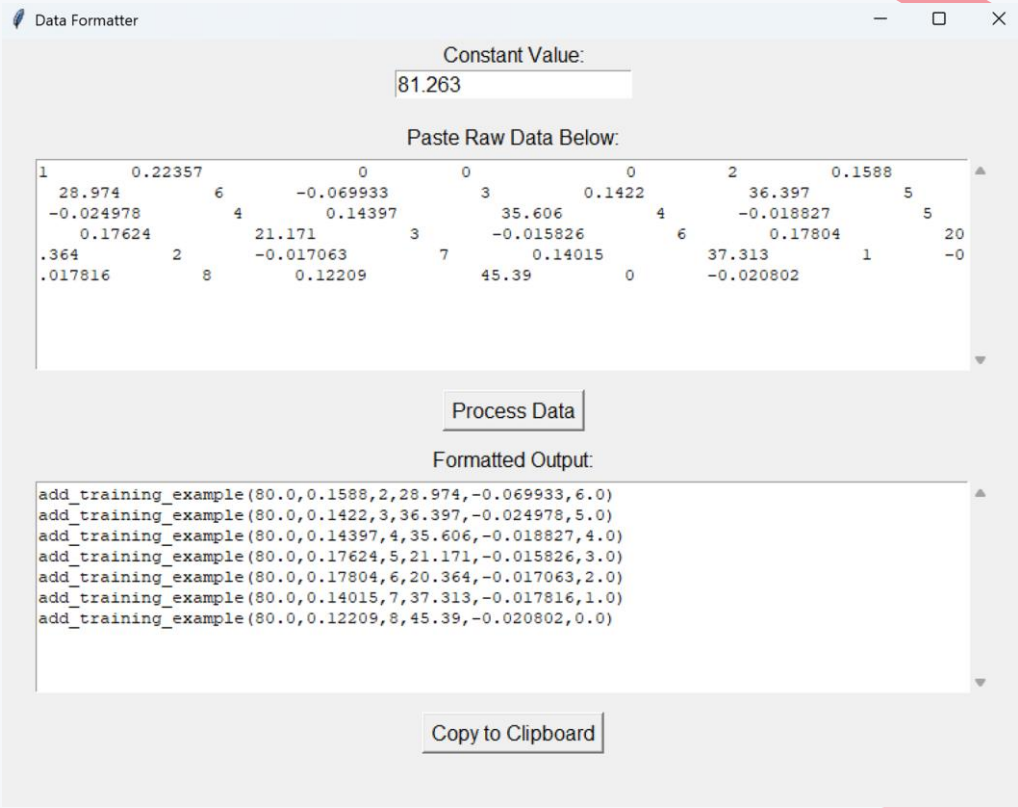






Stage 4

- Made a tool to take MATLAB output directly into lines to place into model



Our features were all calculated mathematically, so thankfully we could easily do it with matlab and python

Stage 4

Data Cleanup

FEATURES WE USED TO TRAIN (INPUTS):

1. Velocity of last (latest) rep
2. Reps completed
3. Load Percentage (How much percent of each person's max)
4. Actual Reps in Reserve

- Originally we had velocity loss percentage as well (change in velocity from previous rep to current rep), as well as slope of the best fit line of the peak velocities, but these were so spread out among the testers that we had to remove them

- One issue originally was the lack of spread in actual RIR

```
    X_new = pd.DataFrame([{
        "load_percent": load_percent,
        "mean_velocity_last_rep": vel_last_rep,
        "velocity_loss": vel_loss,
        "reps_completed": reps_completed,
        "slope": slope
    }])

    raw_rir = model.predict(X_new)[0]
    rounded_rir = round(raw_rir)
    print(f"✅ Predicted RIR (): {rounded_rir}")

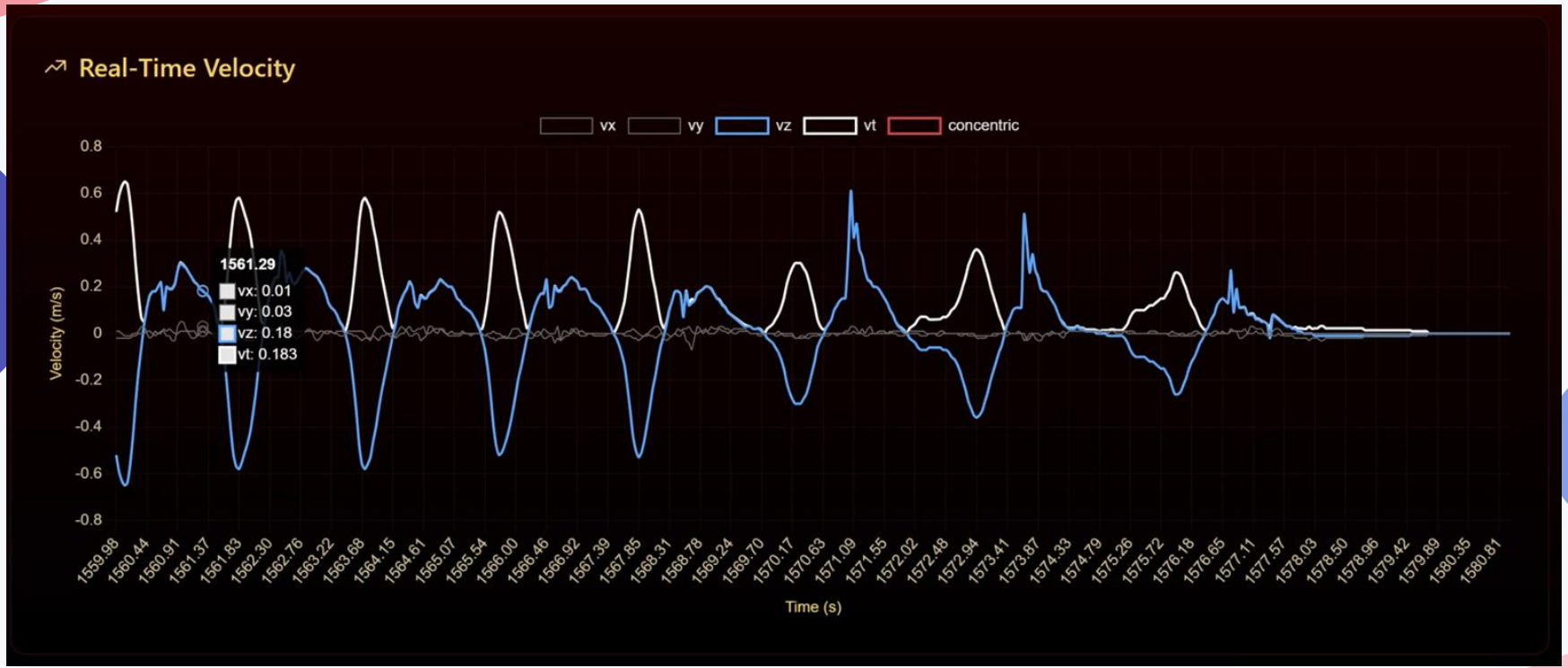
    return rounded_rir


# -------- ENTER YOUR INPUTS HERE -------- #
predict_rir(
    load_percent=100,
    vel_last_rep=0.77,
    vel_loss= 12.3,
    reps_completed = 2,
    slope= -0.0000001
)
```

CONCLUSIONS

Stage 4

- In the future, we can improve our arduino code and filtering to still remove gravity while allowing for quick velocity calculations
- Improve rep detection algorithm
- Use more features to provide insights into form and whatnot
- Get more data and train and better model


Real-Time Velocity
vx
vy
vz
vt
concentric
0.8
0.6
0.4
0.2
0
-0.2
-0.4
-0.6
-0.8
Velocity (m/s)
1561.29
vx: 0.01
vy: 0.03
vz: 0.18
vt: 0.183
1559.98
1560.44
1560.91
1561.37
1561.83
1562.30
1562.76
1563.22
1563.68
1564.15
1564.61
1565.07
1565.54
1566.00
1566.46
1566.92
1567.39
1567.85
1568.31
1568.78
1569.24
1569.70
1570.17
1570.63
1571.09
1571.55
1572.02
1572.48
1572.94
1573.41
1573.87
1574.33
1574.79
1575.26
1575.72
1576.18
1576.65
1577.11
1577.57
1578.03
1578.50
1578.96
1579.42
1579.89
1580.35
1580.81
Time (s)

Reps Completed	Velocity Loss	Mean Velocity (Last Rep)	Estimated RIR
8	64.9%	0.15	3

AI Coach Insight

Good working range — maintain load and focus on bar speed and technique.

Confidence: **medium**

Rep Velocity Analysis

